OCA EXAM 8

CHAPTER 6 – Exceptions

By İbrahim Kürce

Brief of OCA: Oracle Certified Associate Java SE 8 Programmer I Study Guide: Exam 1Z0-808

Jeanne Boyarsky, Scott Selikoff

Understanding Exceptions

- The OCA exam covers only the basics of working with exceptions. The rest are on the OCP exam.
- A program can fail for just about any reason. Here are just a few possibilities:

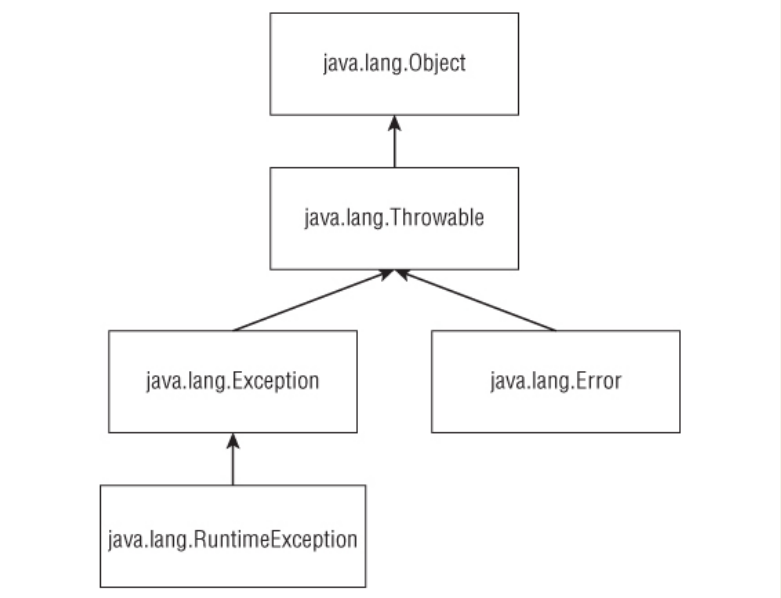
1. The code tries to connect to a website, but the Internet connection is down.
2. You made a coding mistake and tried to access an invalid index in an array.
3. One method calls another with a value that the method doesn't support.

The Role of Exceptions

- An **exception** is Java's way of saying, “I give up. I don't know what to do right now. You deal with it.” When you write a method, you can either deal with the exception or make it the calling code's problem.
- When you write more advanced programs, you'll need to deal with failures in accessing files, networks, and outside services.

Understanding Exception Types

- An exception is an event that alters program flow.
- Java has a **Throwable** superclass for all objects that represent these events.



- Error means something went so horribly wrong that your program should not attempt to recover from it.

Understanding Exception Types

- A **runtime exception** is defined as the RuntimeException class and its subclasses. Runtime exceptions tend to be unexpected but not necessarily fatal. For example, accessing an invalid array index is unexpected. Runtime exceptions are also known as **unchecked exceptions**.
- A **checked exception** includes Exception and all subclasses that do not extend RuntimeException. Checked exceptions tend to be more anticipated— for example, trying to read a file that doesn't exist.
- Java has a rule called **the handle or declare rule**. For checked exceptions, Java requires the code to either handle them or declare them in the method signature.

```
void fall() throws Exception {
  throw new Exception();
}
```

Throwing an Exception

- Any Java code can **throw an exception**; this includes code you write. For the OCP exam, you'll learn how to create your own exception classes. The OCA exam is limited to exceptions that someone else has created.
- On the exam, you will see two types of code that result in an exception. The first is code that's wrong.

```
String[] animals = new String[0];
System.out.println(animals[0]);
```

- The second way for code to result in an exception is to explicitly request Java to throw one.

```
throw new Exception();
throw new Exception("Ow! I fell.");
```

Throwing an Exception

- These rules are very important.

Type	How to recognize	Okay for program to catch?	Is program required to handle or declare?
Runtime exception	Subclass of `RuntimeException`	Yes	No
Checked exception	Subclass of `Exception` but not subclass of `RuntimeException`	Yes	Yes
Error	Subclass of `Error`	No	No

Using a try Statement

- Java uses a **try** statement to separate the logic that might throw an exception from the logic to handle that exception.
- The code in the try block is run **normally**. If any of the statements throw an exception that can be caught by the exception type listed in the catch block, the **try block stops** running and execution **goes to the catch** statement. If none of the statements in the try block throw an exception that can be caught, the **catch clause is not run**.

```
3: void explore() {
4:   try {
5:    fall();
6:    System.out.println("never get here");
7:   } catch (RuntimeException e) {
8:     getUp();
9:   }
10:  seeAnimals();
11: }
12: void fall() {  throw new RuntimeException(); }
```

Using a try Statement

- Try statements are like methods in that the curly braces are required even if there is only one statement inside the code blocks.

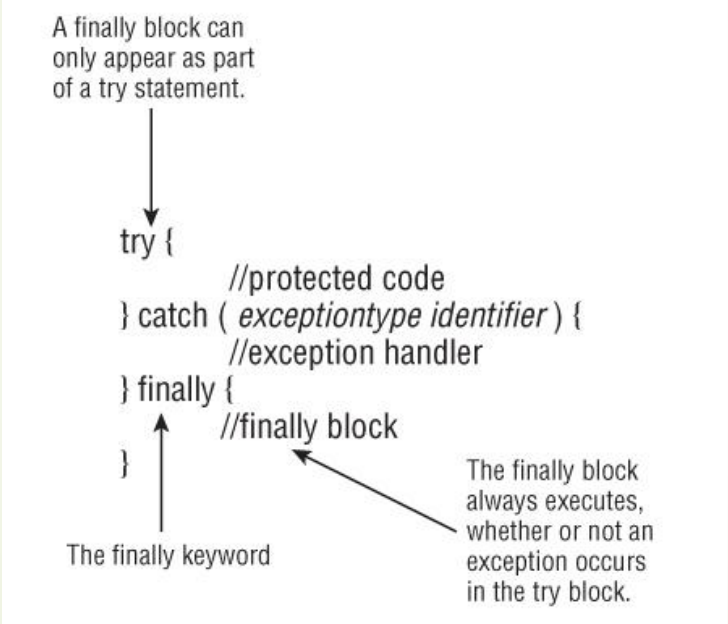
```
try  // DOES NOT COMPILE
  fall();
catch (Exception e)
  System.out.println("get up");
```

```
try {
  fall();
} catch (Exception e) {
  System.out.println("get up");
}
```

```
try {// DOES NOT COMPILE
  fall();
}
```

Adding a finally Block

- The try statement also lets you run code at the end with a finally clause regardless of whether an exception is thrown.



- If an exception is thrown, the finally block is run after the catch block. If no exception is thrown, the finally block is run after the try block completes.

Adding a finally Block

- The exam will try to trick you with missing clauses or clauses in the wrong order.

```
try { // DOES NOT COMPILE
  fall();
} finally {
  System.out.println("all better");
} catch (Exception e) {
  System.out.println("get up");
}
```

- Catch is not required if finally is present.

```
try {
  fall();
} finally {
  System.out.println("all better");
}
```

Adding a finally Block

- What this code outputs:

```
String s = "";
try {
  s += "t";
} catch(Exception e) {
  s += "c";
} finally {
  s += "f";
}
s += "a";
System.out.print(s);
```

Catching Various Types of Exceptions

- Creating your own exceptions is not on the OCA exam, but it is on the OCP exam.
- You must be able to recognize if the exception is a checked or an unchecked exception.
- You need to determine if any of the exceptions are subclasses of the others.

```
class AnimalsOutForAWalk extends RuntimeException { }
class ExhibitClosed extends RuntimeException { }
class ExhibitClosedForLunch extends ExhibitClosed { }
```

- There are three custom exceptions.

Catching Various Types of Exceptions

- There are three possibilities for when this code is run. If seeAnimal() doesn't throw an exception, nothing is printed out. If the animal is out for a walk, only the first catch block runs. If the exhibit is closed, only the second catch block runs.

```
public void visitPorcupine() {
  try {
    seeAnimal();
  } catch (AnimalsOutForAWalk e) {// first catch block
    System.out.print("try back later");
  } catch (ExhibitClosed e) {// second catch block
    System.out.print("not today");

  }
}
```

Catching Various Types of Exceptions

- A rule exists for the order of the catch blocks. Java looks at them in the order they appear. If it is impossible for one of the catch blocks to be executed, a compiler error about unreachable code occurs.

```
  try {
    seeAnimal();
  } catch (ExhibitClosedForLunch e) {// subclass exception
    System.out.print("try back later");
  } catch (ExhibitClosed e) {// superclass exception
    System.out.print("not today");
  }
}
```

Catching Various Types of Exceptions

```
public void visitMonkeys() {
  try {
    seeAnimal();
  } catch (ExhibitClosed e) {
    System.out.print("not today");
  } catch (ExhibitClosedForLunch e) {// DOES NOT COMPILE
    System.out.print("try back later");
  }
}
```

Catching Various Types of Exceptions

```
public void visitSnakes() {
  try {
    seeAnimal();
  } catch (RuntimeException e) {
    System.out.print("runtime exception");
  } catch (ExhibitClosed e) {// DOES NOT COMPILE
    System.out.print("not today");
  } catch (Exception e) {
    System.out.print("exception");
  }
}
```

Throwing a Second Exception

So far, we've limited ourselves to one try statement in each example. However, a catch or finally block can have any valid Java code in it—including another try statement.

```
public static void main(String[] args) {
  FileReader reader = null;
  try {
    reader = read();
  } catch (IOException e) {
    try {
     if (reader != null)  reader.close();
    } catch (IOException inner) {
    }
  }
}
private static FileReader read() throws IOException {
  // CODE GOES HERE
}
```

Throwing a Second Exception

Most of the examples you see with exception handling on the exam are abstract. They use letters or numbers to make sure you understand the flow. (This is true for the OCA exam. It will change a bit on the OCP exam.)

```
26: try {
27:   throw new RuntimeException();
28: } catch (RuntimeException e) {
29:   throw new RuntimeException();
30: } finally {
31:   throw new Exception();
32: }
```

Throwing a Second Exception

```
30: public String exceptions() {
31:   String result = "";
32:   String v = null;
33:   try {
34:      try {
35:        result += "before";
36:        v.length();
37:        result += "after";
38:      } catch (NullPointerException e) {
39:       result += "catch";
40:       throw new RuntimeException();
41:      } finally {
42:       result += "finally";
43:       throw new Exception();
44:      }
45:   } catch (Exception e) {
46:     result += "done";
47:   }
48:   return result;
49: }
```

Recognizing Common Exception Types

- You need to recognize three types of exceptions for the OCA exam:
  - runtime exceptions
  - checked exceptions, and
  - errors.

Runtime Exceptions

Runtime exceptions extend RuntimeException. They don't have to be handled or declared. They can be thrown by the programmer or by the JVM.

- **ArithmeticException** Thrown by the JVM when code attempts to divide by zero
- **ArrayIndexOutOfBoundsException** Thrown by the JVM when code uses an illegal index to access an array
- **ClassCastException** Thrown by the JVM when an attempt is made to cast an exception to a subclass of which it is not an instance
- **IllegalArgumentException** Thrown by the programmer to indicate that a method has been passed an illegal or inappropriate argument
- **NullPointerException** Thrown by the JVM when there is a null reference where an object is required
- **NumberFormatException** Thrown by the programmer when an attempt is made to convert a string to a numeric type but the string doesn't have an appropriate format

ArithmeticException

- Trying to divide an int by zero gives an undefined result. When this occurs, the JVM will throw an ArithmeticException.
- int answer = 11 / 0;
- Exception in thread "main" java.lang.ArithmeticException: / by zero.

ArrayIndexOutOfBoundsException

- Array indexes start with 0 and go up to 1 less than the length of the array.

```
int[] countsOfMoose = new int[3];
System.out.println(countsOfMoose[-1]);
```

- The problem is that the for loop should have < instead of <=.

```
int total = 0;
int[] countsOfMoose = new int[3];
for (int i = 0; i <= countsOfMoose.length; i++)
  total += countsOfMoose[i];
```

ClassCastException

- Java tries to protect you from impossible casts. This code doesn't compile because Integer is not a subclass of String:

```
String type = "moose";
Integer number = (Integer) type;  // DOES NOT COMPILE
```

- When the cast fails at runtime, Java will throw a ClassCastException:

```
String type = "moose";
Object obj = type;
Integer number = (Integer) obj;
```

IllegalArgumentException

- IllegalArgumentException is a way for your program to protect itself.

```
6:  public void setNumberEggs(int numberEggs) {// setter
7:    if (numberEggs >= 0) // guard condition
8:      this.numberEggs = numberEggs;
9:  }
```

- This code works, but we don't really want to ignore the caller's request when they tell us a Swan has –2 eggs. We want to tell the caller that something is wrong. Exceptions are an efficient way to do this.

```
public static void setNumberEggs(int numberEggs) {
  if (numberEggs < 0)
    throw new IllegalArgumentException(
        "# eggs must not be negative");
  this.numberEggs = numberEggs;
}
```

NullPointerException

- Instance variables and methods must be called on a non-null reference. If the reference is null, the JVM will throw a NullPointerException.

```
String name;

public void printLength() throws NullPointerException {
  System.out.println(name.length());
}
```

NumberFormatException

- Java provides methods to convert strings to numbers. When these are passed an invalid value, they throw a NumberFormatException.
- NumberFormatException is a subclass of IllegalArgumentException.

```
Integer.parseInt("abc");
```

Checked Exceptions

- Checked exceptions have Exception in their hierarchy but not RuntimeException. They must be handled or declared. They can be thrown by the programmer or by the JVM. Common runtime exceptions include the following:
- **FileNotFoundException** Thrown programmatically when code tries to reference a file that does not exist
- **IOException** Thrown programmatically when there's a problem reading or writing a file
- For the OCA exam, you only need to know that these are checked exceptions. Also keep in mind that FileNotFoundException is a subclass of IOException.

Errors

- Errors extend the Error class. They are thrown by the JVM and should not be handled or declared. Errors are rare, but you might see these:
- **ExceptionInInitializerError** Thrown by the JVM when a static initializer throws an exception and doesn't handle it
- **StackOverflowError** Thrown by the JVM when a method calls itself too many times (this is called *infi nite recursion* because the method typically calls itself without end)
- **NoClassDefFoundError** Thrown by the JVM when a class that the code uses is available at compile time but not runtime

ExceptionInInitializerError

- Java runs static initializers the first time a class is used. If one of the static initializers throws an exception, Java can't start using the class. It declares defeat by throwing an ExceptionInInitializerError.

```
static {
  int[] countsOfMoose = new int[3];
  int num = countsOfMoose[-1];
}
public static void main(String[] args) { }
```

- Exception in thread "main" java.lang.ExceptionInInitializerError
  Caused by: java.lang.ArrayIndexOutOfBoundsException: -1

StackOverflowError

- When Java calls methods, it puts parameters and local variables on the stack. After doing this a very large number of times, the stack runs out of room and overflows. This is called a StackOverflowError.
- Most of the time, this error occurs when a method calls itself.

```
public static void doNotCodeThis(int num) {
  doNotCodeThis(1);
}
```

- ***NoClassDefFoundError:*** This error won't show up in code on the exam—you just need to know that it is an error. NoClassDefFoundError occurs when Java can't find the class at runtime.

Calling Methods That Throw Exceptions

- Checked exceptions must be handled or declared.

```
private static void eatCarrot() throws NoMoreCarrotsException {
}
```

- You might have noticed that eatCarrot() didn't actually throw an exception; it just declared that it could. This is enough for the compiler to require the caller to handle or declare the exception.

```
public static void main(String[] args)
  throws NoMoreCarrotsException {// declare exception
  eatCarrot();
}
public static void main(String[] args) {
  try {
    eatCarrot();
  } catch (NoMoreCarrotsException e ) {// handle exception
    System.out.print("sad rabbit");
  }
}
```

Calling Methods That Throw Exceptions

- Declaring an unused exception isn't considered unreachable code. It gives the method the option to change the implementation to throw that exception in the future.

```
public void bad() {
  try {
    eatCarrot();
  } catch (NoMoreCarrotsException e ) {// DOES NOT COMPILE
    System.out.print("sad rabbit");
  }
}
public void good() throws NoMoreCarrotsException {
  eatCarrot();
}
private static void eatCarrot() { }
```

Subclasses

- When a class overrides a method from a superclass or implements a method from an interface, it's not allowed to add new checked exceptions to the method signature.

```
class CanNotHopException extends Exception { }
class Hopper {
  public void hop() { }
}
class Bunny extends Hopper {
  public void hop() throws CanNotHopException { } // DOES NOT COMPILE
}
```

- Java knows hop() isn't allowed to throw any checked exceptions because the superclass Hopper doesn't declare any.

Subclasses

- A subclass is allowed to declare fewer exceptions than the superclass or interface. This is legal because callers are already handling them.

```
class Hopper {
  public void hop() throws CanNotHopException { }
}
class Bunny extends Hopper {
  public void hop()  { }
}
```

- A class is allowed to declare a subclass of an exception type. The superclass or interface has already taken care of a broader type.

```
class Hopper {
  public void hop() throws Exception { }
}
class Bunny extends Hopper {
  public void hop() throws CanNotHopException { }
}
```

Subclasses

- This rule applies only to checked exceptions. The following code is legal because it has a runtime exception in the subclass's version:

```
class Hopper {
  public void hop() { }
}

class Bunny extends Hopper {
  public void hop() throws IllegalStateException { }
}
```

- The reason that it's okay to declare new runtime exceptions in a subclass method is that the declaration is redundant. Methods are free to throw any runtime exceptions they want without mentioning them in the method declaration.

Printing an Exception

- There are three ways to print an exception

```
5: public static void main(String[] args) {
6:    try {
7:     hop();
8:    } catch (Exception e) {
9:      System.out.println(e);
10:     System.out.println(e.getMessage());
11:     e.printStackTrace();
12:   }
13: }
14: private static void hop() {
15:   throw new RuntimeException("cannot hop");
16: }
```

- The stack trace is usually the most helpful one because it shows where the exception occurred in each method that it passed through. On the OCA exam, you will mostly see **the first approach**. This is because the exam often shows code snippets.

Exam Essentials

- **Differentiate between checked and unchecked exceptions.** Unchecked exceptions are also known as runtime exceptions and are subclasses of java.lang.RuntimeException. All other subclasses of java.lang.Exception are checked exceptions.
- **Understand the flow of a** try **statement.** A try statement must have a catch or a finally block. Multiple catch blocks are also allowed, provided no superclass exception type appears in an earlier catch block than its subclass. The finally block runs last regardless of whether an exception is thrown.
- **Identify whether an exception is thrown by the programmer or the JVM.** Illegal ArgumentException and NumberFormatException are commonly thrown by the programmer. Most of the other runtime exceptions are typically thrown by the JVM.
- **Declare methods that declare exceptions.** The throws keyword is used in a method declaration to indicate an exception might be thrown. When overriding a method, the method is allowed to throw fewer exceptions than the original version.
- **Recognize when to use throw versus throws.** The throw keyword is used when you actually want to throw an exception—for example, throw new RuntimeException(). The throws keyword is used in a method declaration.

Review Questions

1. Which of the following statements are true? (Choose all that apply)
   - **A.** Runtime exceptions are the same thing as checked exceptions.
   - **B.** Runtime exceptions are the same thing as unchecked exceptions.
   - **C.** You can declare only checked exceptions.
   - **D.** You can declare only unchecked exceptions.
   - **E.** You can handle only `Exception` subclasses.

Ans: **B**

2. Which of the following pairs fill in the blanks to make this code compile? (Choose all that apply)

```
7: public void ohNo() _____ Exception {
8:   _____________ Exception();
9: }
```

A. On line 7, fill in `throw`

B. On line 7, fill in `throws`

C. On line 8, fill in `throw`

D. On line 8, fill in `throw new`

E. On line 8, fill in `throws`

F. On line 8, fill in `throws new`

Ans: B, D

3. When are you required to use a `finally` block in a regular `try` statement (not a `try-with-resources`)?

A. Never.

B. When the program code doesn't terminate on its own.

C. When there are no `catch` blocks in a `try` statement.

D. When there is exactly one `catch` block in a `try` statement.

E. When there are two or more `catch` blocks in a `try` statement.

Ans: C

4. Which exception will the following throw?

```
Object obj = new Integer(3);
String str = (String) obj;
System.out.println(str);
```

A. `ArrayIndexOutOfBoundsException`

B. `ClassCastException`

C. `IllegalArgumentException`

D. `NumberFormatException`

E. None of the above.

Ans: **B**

5. Which of the following exceptions are thrown by the JVM? (Choose all that apply)

A. `ArrayIndexOutOfBoundsException`

B. `ExceptionInInitializerError`

C. `java.io.IOException`

D. `NullPointerException`

E. `NumberFormatException`

Ans: **A, B, D**

6. What will happen if you add the statement `System.out.println(5 / 0);` to a working `main()` method?

A. It will not compile.

B. It will not run.

C. It will run and throw an `ArithmeticException`.

D. It will run and throw an `IllegalArgumentException`.

E. None of the above.

Ans: C

7. What is printed besides the stack trace caused by the `NullPointerException` from line 16?

```
1: public class DoSomething {
2:   public void go() {
3:     System.out.print("A");
4:     try {
5:         stop();
6:     } catch (ArithmeticException e) {
7:         System.out.print("B");
8:     } finally {
9:         System.out.print("C");
10:    }
11:    System.out.print("D");
12:  }
13:  public void stop() {
14:    System.out.print("E");
15:    Object x = null;
16:    x.toString();
17:    System.out.print("F");
18:  }
19:  public static void main(String[] args) {
20:    new DoSomething().go();
21:  }
22: }
```

A. AE

B. AEBCD

C. AEC

D. AECD

E. No output appears other than the stack trace.

Ans: C

8. What is the output of the following snippet, assuming a and b are both 0?

```
try {
  return a / b;
} catch (RuntimeException e) {
  return -1;
} catch (ArithmeticException e) {
  return 0;
} finally {
  System.out.print("done");
}
```

A. -1

B. 0

C. done-1

D. done0

E. The code does not compile.

F. An uncaught exception is thrown.

Ans: E

9. What is the output of the following program?

```
1: public class Laptop {
2:   public void start() {
3:    try {
4:      System.out.print("Starting up ");
5:      throw new Exception();
6:    } catch (Exception e) {
7:       System.out.print("Problem ");
8:       System.exit(0);
9:    } finally {
10:      System.out.print("Shutting down ");
11:   }
12:  }
13:  public static void main(String[] args) {
14:    new Laptop().start();
15:  } }
```

A. `Starting up`

B. `Starting up Problem`

C. `Starting up Problem Shutting down`

D. `Starting up Shutting down`

E. The code does not compile.

F. An uncaught exception is thrown.

Ans: **B**

10. What is the output of the following program?

```
1:  public class Dog {
2:    public String name;
3:    public void parseName() {
4:      System.out.print("1");
5:      try {
6:        System.out.print("2");
7:        int x = Integer.parseInt(name);
8:        System.out.print("3");
9:      } catch (NumberFormatException e) {
10:       System.out.print("4");
11:     }
12:   }
13:   public static void main(String[] args) {
14:     Dog leroy = new Dog();
15:     leroy.name = "Leroy";
16:     leroy.parseName();
17:     System.out.print("5");
18:   } }
```

A. 12

B. 1234

C. 1235

D. 124

E. 1245

F. The code does not compile.

G. An uncaught exception is thrown.

Ans: E

11. What is the output of the following program?

```
1:  public class Cat {
2:    public String name;
3:    public void parseName() {
4:      System.out.print("1");
5:      try {
6:        System.out.print("2");
7:        int x = Integer.parseInt(name);
8:        System.out.print("3");
9:      } catch (NullPointerException e) {
10:       System.out.print("4");
11:     }
12:     System.out.print("5");
13:   }
14:   public static void main(String[] args) {
15:     Cat leo = new Cat();
16:     leo.name = "Leo";
17:     leo.parseName();
18:     System.out.print("6");
19:   }
20: }
```

A. 12, followed by a stack trace for a `NumberFormatException`

B. 124, followed by a stack trace for a `NumberFormatException`

C. 12456

D. 12456

E. 1256, followed by a stack trace for a `NumberFormatException`

F. The code does not compile.

G. An uncaught exception is thrown.

Ans: A

12. What is printed by the following? (Choose all that apply)

```
1:  public class Mouse {
2:    public String name;
3:    public void run() {
4:      System.out.print("1");
5:      try {
6:        System.out.print("2");
7:        name.toString();
8:        System.out.print("3");
9:      } catch (NullPointerException e) {
10:       System.out.print("4");
11:       throw e;
12:     }
13:     System.out.print("5");
14:   }
15:   public static void main(String[] args) {
16:     Mouse jerry = new Mouse();
17:     jerry.run();
18:     System.out.print("6");
19:   } }
```

A. 1
B. 2
C. 3
D. 4
E. 5
F. 6
G. The stack trace for a `NullPointerException`

Ans: **A, B, D, G**

13. Which of the following statements are true? (Choose all that apply)

A. You can declare a method with `Exception` as the return type.

B. You can declare any subclass of `Error` in the `throws` part of a method declaration.

C. You can declare any subclass of `Exception` in the `throws` part of a method declaration.

D. You can declare any subclass of `Object` in the `throws` part of a method declaration.

E. You can declare any subclass of `RuntimeException` in the `throws` part of a method declaration.

Ans: **A, B, C, E**

14. Which of the following can be inserted on line 8 to make this code compile? (Choose all that apply)

```
7: public void ohNo() throws IOException {
8:   // INSERT CODE HERE
9: }
```

A. `System.out.println("it's ok");`

B. `throw new Exception();`

C. `throw new IllegalArgumentException();`

D. `throw new java.io.IOException();`

E. `throw new RuntimeException();`

Ans: **A, C, D, E**

15. Which of the following are unchecked exceptions? (Choose all that apply)

A. `ArrayIndexOutOfBoundsException`

B. `IllegalArgumentException`

C. `IOException`

D. `NumberFormatException`

E. Any exception that extends `RuntimeException`

F. Any exception that extends `Exception`

Ans: **A, B, D, E**

16. Which scenario is the best use of an exception?

A. An element is not found when searching a list.

B. An unexpected parameter is passed into a method.

C. The computer caught fire.

D. You want to loop through a list.

E. You don't know how to code a method.

Ans: **B**

17. Which of the following can be inserted into `Lion` to make this code compile? (Choose all that apply)

```
class HasSoreThroatException extends Exception {}
class TiredException extends RuntimeException {}
interface Roar {
  void roar() throws HasSoreThroatException;
}
class Lion implements Roar {// INSERT CODE HERE
}
```

A. `public void roar(){}`

B. `public void roar() throws Exception{}`

C. `public void roar() throws HasSoreThroatException{}`

D. `public void roar() throws IllegalArgumentException{}`

E. `public void roar() throws TiredException{}`

Ans: A, C, D, E

18. Which of the following are true? (Choose all that apply)

A. Checked exceptions are allowed to be handled or declared.

B. Checked exceptions are required to be handled or declared.

C. Errors are allowed to be handled or declared.

D. Errors are required to be handled or declared.

E. Runtime exceptions are allowed to be handled or declared.

F. Runtime exceptions are required to be handled or declared.

Ans: **A, B, C, E**

19. Which of the following can be inserted in the blank to make the code compile? (Choose all that apply)

```
public static void main(String[] args) {
  try {
    System.out.println("work real hard");
  } catch (_________________ e) {
  } catch (RuntimeException e) {
  }
}
```

A. `Exception`

B. `IOException`

C. `IllegalArgumentException`

D. `RuntimeException`

E. `StackOverflowError`

F. None of the above.

Ans: C, E

20. What does the output of the following contain? (Choose all that apply)

```
public static void main(String[] args) {
  System.out.print("a");
  try {
    System.out.print("b");
    throw new IllegalArgumentException();
  } catch (IllegalArgumentException e) {
    System.out.print("c");
    throw new RuntimeException("1");
  } catch (RuntimeException e) {
    System.out.print("d");
    throw new RuntimeException("2");
  } finally {
    System.out.print("e");
    throw new RuntimeException("3");
  }
}
```

A. abce

B. abde

C. An exception with the message set to "1"

D. An exception with the message set to "2"

E. An exception with the message set to "3"

F. Nothing; the code does not compile.

Ans: A, E